



Patrones creacionales: quién hace el *new*

Qué es un patrón de diseño

Un patrón de diseño es una solución con nombre a un problema de diseño que se repite. No es un algoritmo ni código para copiar. Es una descripción de cómo organizar clases y objetos para resolver un problema que muchos programadores enfrentaron por separado, hasta que alguien la escribió, le puso nombre y la publicó. Ya se usó uno, Observer, cuando el modelo tenía que avisar a varias ventanas. La solución fue una lista de observadores, una interfaz y un *for*, y lo que se ganó con el nombre es poder decir “el modelo es observable” y que todo el equipo entienda la estructura sin ver el código.

El catálogo de referencia es el libro de Gamma, Helm, Johnson y Vlissides de 1994, conocido como el libro de la “banda de los cuatro”. Describe veintitrés patrones y los clasifica según su propósito en tres grupos. Los creacionales se ocupan de cómo se crean los objetos. Los estructurales, de cómo se combinan clases y objetos en estructuras más grandes. Los de comportamiento, de cómo se reparten responsabilidades y se comunican. Para leerlos hoy, la referencia más accesible es Refactoring Guru (<https://refactoring.guru/es/design-patterns>), un sitio gratuito en castellano con la explicación, los diagramas y el código en Java de cada uno de los veintitrés, y su versión en libro, *Dive Into Design Patterns*, de Alexander Shvets. Esta pre clase es la primera de las tres y presenta tres patrones creacionales. Cada uno se describe de la misma manera, con el problema que resuelve, la solución en general con sus participantes, un ejemplo, y sus ventajas y desventajas.

Los patrones creacionales

En un programa orientado a objetos casi todo el código trabaja con abstracciones. Un método recibe un *Pago* y no le importa si es en efectivo o con tarjeta, y una lista de *Producto* se recorre sin preguntar de qué tipo es cada elemento. Hay un único lugar donde eso no se puede sostener, y es el *new*. Para crear un objeto hay que nombrar su clase concreta.

Eso no es un problema cuando el *new* está en un solo lugar. Lo es cuando la decisión de qué clase crear se repite por todo el programa, porque cada lugar que la toma queda atado a las clases concretas y a la forma de construirlas. Agregar un tipo obliga a encontrar y abrir todos esos lugares. Cambiar la manera de construir un objeto, también. Y el polimorfismo, que resuelve este problema en el uso de los objetos, no puede resolverlo en la creación, porque antes del *new* no existe ningún objeto al que preguntarle nada.

Los patrones creacionales separan la decisión de qué crear y cómo crearlo del código que usa lo creado. Esta pre clase presenta tres. Factory, en sus dos formas, resuelve quién decide la clase concreta. Builder resuelve cómo construir objetos con muchos datos. Singleton resuelve cómo garantizar que un objeto exista una sola vez.

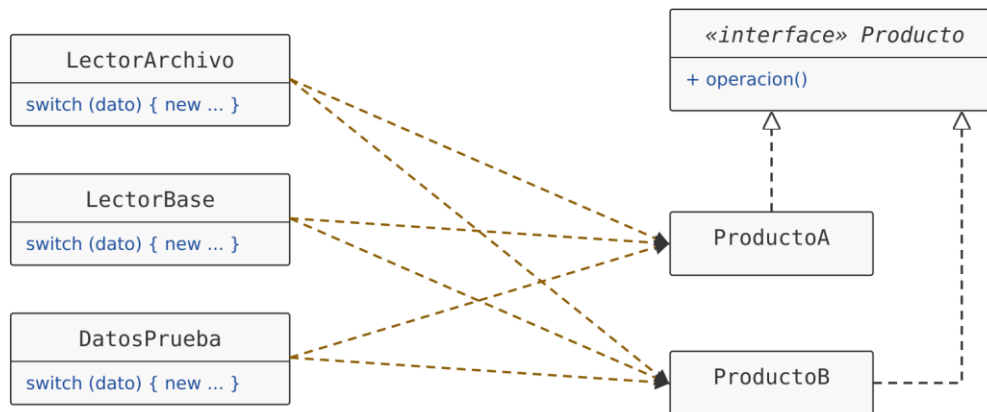
El único código que necesita conocer una clase concreta es el que la crea. Los patrones creacionales reducen ese código a un lugar.

Factory

El problema

La clase a crear depende de un dato que se conoce recién en tiempo de ejecución. Puede ser un texto de un archivo de configuración, la opción que eligió el usuario en un menú o el valor de una columna en la base de datos. La solución directa es un *if* o un *switch* sobre ese dato, con un *new* en cada rama.

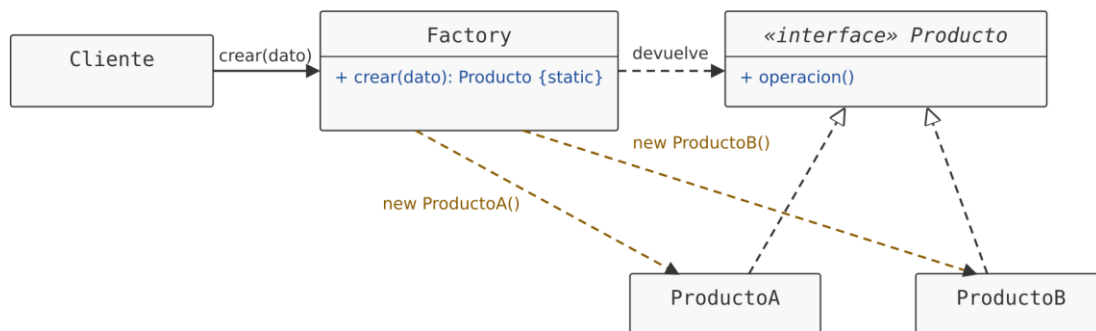
Ese *switch* tiene tres costos. El primero es que se repite, porque el mismo dato se convierte en objeto en más de un lugar, al leer el archivo, al leer la base y al armar datos de prueba. El segundo es que cada copia importa todas las clases concretas, así que el código que debería trabajar solo con la abstracción termina acoplado a cada implementación. El tercero es que un tipo nuevo obliga a encontrar todas las copias, y la que se olvida falla en tiempo de ejecución.



Sin el patrón. Tres lugares deciden la clase concreta y cada uno conoce todos los productos. Seis new para dos productos, y un tipo nuevo obliga a abrir los tres archivos.

La solución

Se escribe una clase, el Factory, cuya única responsabilidad es tomar esa decisión. Tiene un método de creación que recibe el dato y devuelve el objeto, declarado con el tipo de la abstracción. Los participantes son cuatro. El Producto es la interfaz o clase abstracta que el cliente conoce. Los productos concretos son sus implementaciones. El Factory es la clase con el método de creación, y es la única que nombra a los productos concretos. El Cliente le pide el objeto al Factory y trabaja con el Producto.



Estructura de Factory. El cliente conoce al Factory y a Producto. Solo el Factory conoce las clases concretas.

En la forma más simple, el método de creación es estático y contiene el *switch*. El *switch* no desaparece, pero existe una sola vez, y las clases concretas dejan de aparecer en el resto del programa. La forma que también elimina el *switch* reemplaza los *case* por un *Map* de dato a lambda, donde cada tipo se registra con una línea al arrancar.

Ejemplo

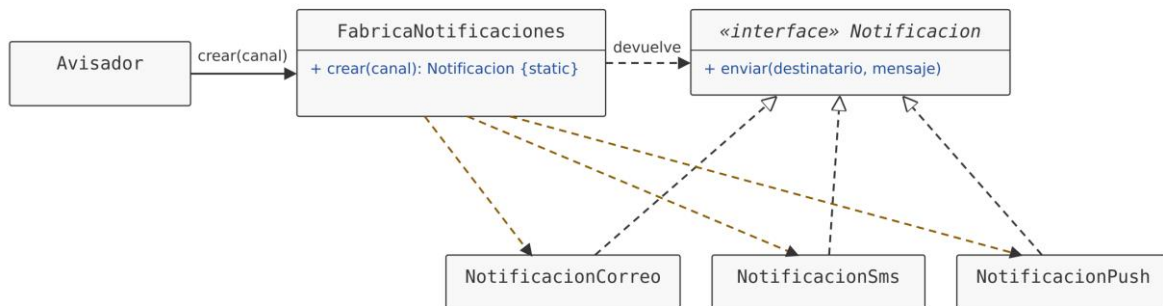
Una aplicación avisa a sus usuarios por el canal que cada uno eligió, que puede ser correo, mensaje de texto o notificación en el teléfono. La preferencia está guardada como texto, y cada canal es una clase que implementa *Notificacion*:

```
public interface Notificacion {
    boolean enviar(String destinatario, String mensaje);
}

public class FabricaNotificaciones {

    public static Notificacion crear(String canal) {
        switch (canal) {
            case "correo": return new NotificacionCorreo();
            case "sms":    return new NotificacionSms();
            case "push":   return new NotificacionPush();
            default:
                throw new IllegalArgumentException("Canal desconocido: " + canal);
        }
    }
}
```

Quien tiene que avisar le pide la notificación al Factory con el canal del usuario y llama a *enviar()*, sin importar ninguna de las tres clases concretas. Cuando se agregue WhatsApp, cambia un *case* en un archivo. El *default* lanza una excepción en vez de devolver *null*, por la misma razón que un constructor con datos inválidos. Un método que no puede cumplir lo que promete tiene que decirlo en el momento, no devolver algo que va a fallar más adelante.



El ejemplo con la estructura de Factory. El Avisador conoce a FabricaNotificaciones y a Notificacion. Los tres canales solo se nombran en el switch.

Ventajas y desventajas

- Las clases concretas se nombran en un solo lugar. El resto del programa depende de la abstracción, y agregar un tipo es cambiar un archivo.
- El Factory puede decidir más que la clase. Puede devolver un objeto ya creado en vez de uno nuevo, o elegir la implementación según la configuración, sin que el cliente lo note.
- El *switch* sigue existiendo, así que el Factory se abre con cada tipo nuevo. Es un lugar controlado, pero es un lugar.
- Un método estático no se puede reemplazar en una prueba. Si una clase llama al Factory por adentro, probarla implica crear los productos de verdad.

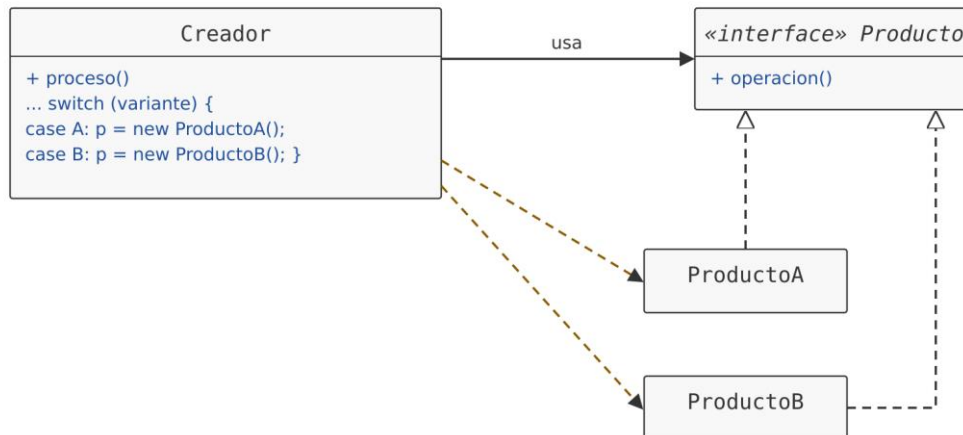
Factory. Un método de creación que recibe el dato y devuelve el Producto. El *switch* por tipo existe, pero existe una vez.

Factory Method

El problema

Una clase hace un proceso de varios pasos, y uno de esos pasos crea un objeto. El proceso es siempre el mismo, pero qué objeto se crea depende de la variante de la clase que se esté usando. No hay un dato que lo decida, porque la decisión ya se tomó al elegir la variante.

Si el proceso se escribe con un *new* de una clase concreta, la clase base queda atada a ella y las variantes tienen que copiar el proceso entero para cambiar una línea. Si se escribe con un *switch*, aparece el mismo problema que antes, y además el proceso conoce a todas sus variantes.

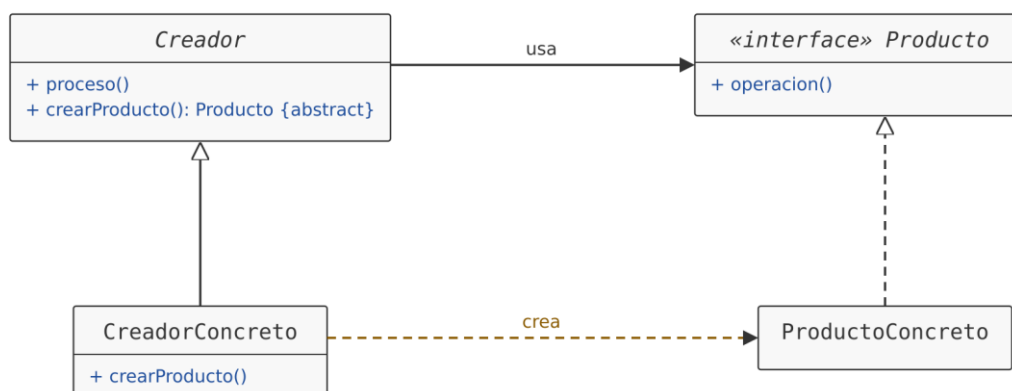


Sin el patrón. El proceso y la decisión de qué crear viven en la misma clase, que se abre con cada variante nueva.

La solución

La clase base escribe el proceso completo y deja el paso de la creación en un método abstracto, el factory method. Cada subclase implementa ese método devolviendo el objeto que le corresponde. La clase base llama al método sin saber qué subclase lo implementa, igual que llama a cualquier otro método abstracto.

Los participantes son cuatro. El Creador es la clase abstracta que escribe el proceso y declara el factory method. Los creadores concretos son sus subclases, y cada una implementa el factory method con un *new* distinto. El Producto es la abstracción que el Creador usa, y los productos concretos son lo que cada creador concreto devuelve.



Estructura de Factory Method. La clase base escribe el proceso y deja el new en un método abstracto, que llama sin saber qué subclase lo implementa.

La diferencia con la forma anterior está en quién elige. En Factory, el cliente pasa un dato y el *switch* decide. En Factory Method no hay dato ni *switch*, porque cada subclase es la decisión.



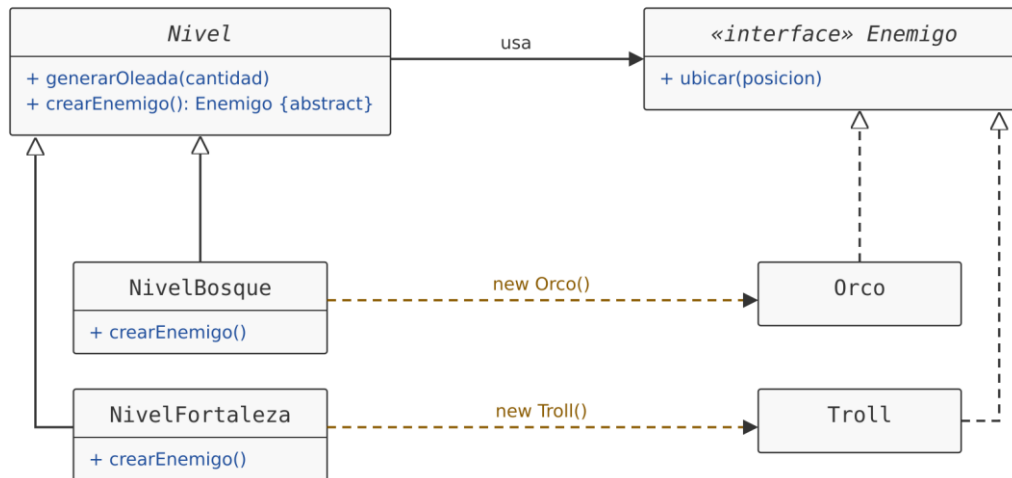
Agregar un tipo es agregar una subclase, y ninguna clase existente se abre. Es lo que pide el principio Open/Closed.

Ejemplo

En un juego, un *Nivel* genera oleadas de enemigos. Calcula cuántos, les busca una posición libre y los agrega al tablero, y ese proceso es igual en todos los niveles. En el bosque se crean orcos y en la fortaleza, trolls:

```
public abstract class Nivel {  
  
    private final List<Enemigo> enemigos = new ArrayList<>();  
  
    public void generarOleada(int cantidad) {  
        for (int i = 0; i < cantidad; i++) {  
            Enemigo enemigo = this.crearEnemigo();    // el factory method  
            enemigo.ubicar(this.posicionLibre());  
            this.enemigos.add(enemigo);  
        }  
    }  
  
    protected abstract Enemigo crearEnemigo();  
}  
  
public class NivelBosque extends Nivel {  
    @Override  
    protected Enemigo crearEnemigo() {  
        return new Orco();  
    }  
}  
  
public class NivelFortaleza extends Nivel {  
    @Override  
    protected Enemigo crearEnemigo() {  
        return new Troll();  
    }  
}
```

El método *generarOleada()* está escrito una vez y no conoce a *Orco* ni a *Troll*. Un nivel nuevo es una clase con un solo método. Quien arma la partida elige el nivel con un *new NivelBosque()*, y de ahí en adelante nadie vuelve a preguntar de qué tipo es.



El ejemplo con la estructura de Factory Method. El método generarOleada() está escrito una vez en Nivel, y cada subclase decide qué enemigo crea.

Ventajas y desventajas

- La clase base no conoce ninguna clase concreta. Agregar un producto es agregar una subclase del Creador, sin tocar el proceso.
- El proceso está escrito una sola vez, y las subclases solo escriben lo que cambia.
- Requiere una jerarquía de creadores paralela a la de productos. Con muchos productos, son muchas clases chicas.
- Si la decisión de qué crear depende de un dato y no de la variante, Factory Method no la resuelve. En ese caso corresponde el Factory anterior.

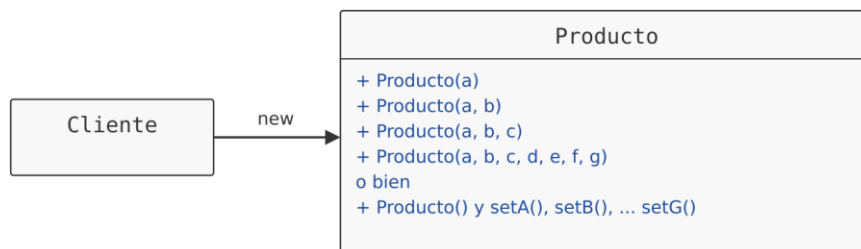
Factory Method. La clase base hace el proceso y la subclase hace el new.

Builder

El problema

Un objeto necesita muchos datos para construirse, y varios son opcionales. Con un constructor por cada combinación, la clase termina con una lista de constructores casi iguales, cada uno con un parámetro más. Con un solo constructor de todos los parámetros, el código que lo llama es una fila de valores donde no se sabe qué es cada uno, los *null* y los ceros marcan lo que no se usa, y dos parámetros del mismo tipo se pasan al revés sin que el compilador diga nada.

La alternativa de un constructor vacío y un setter por atributo tiene otro costo. Entre el *new* y el último *set* el objeto está incompleto, y nada impide usarlo así. La clase no puede validar los datos en conjunto, porque nunca sabe cuándo terminaron de llegar.

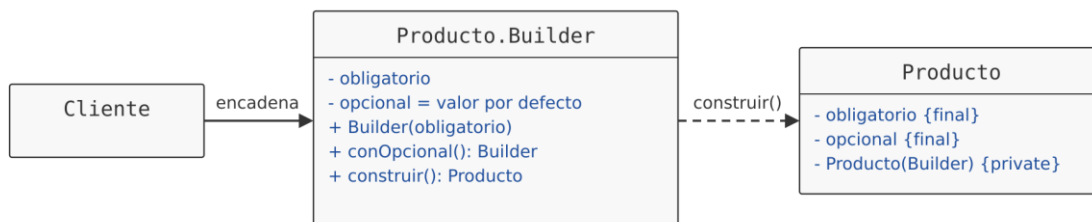


`new Producto("x", null, 0, "", true, null, 0)`
`new Producto(); p.setA("x"); ... p.setG(0);`

Sin el patrón. Un constructor por combinación, o uno de todos los parámetros donde no se sabe qué es cada valor, o un objeto que se completa con setters y está incompleto entre el new y el último set.

La solución

La construcción se separa en dos objetos. El Builder junta los datos de a uno, con un método por atributo que devuelve el propio Builder, de modo que las llamadas se encadenan y se leen. Los datos obligatorios se piden en el constructor del Builder, así no se pueden omitir, y los opcionales tienen un valor por defecto. Al final, un método *construir()* llama al constructor del Producto, que recibe el Builder completo, valida todo de una vez y asigna. El constructor del Producto es privado, así que la única forma de obtener uno es a través del Builder, y el Producto nace completo o no nace.



Estructura de Builder. El Builder junta los datos de a uno, cada método devuelve this, y el Producto se crea al final, completo y validado por su constructor.

En Java el Builder se escribe como una clase anidada *static* dentro del Producto, para que el constructor privado del Producto pueda leer los atributos del Builder sin getters. El libro original describe una versión más general, con un Director que decide el orden de los pasos y varios Builders para el mismo proceso. La versión anidada es la que se usa en la práctica.

Ejemplo

Una reserva de hotel tiene habitación y cantidad de noches, que son obligatorias, y desayuno, cochera y cuna, que son opcionales:



```
public class Reserva {

    private final int habitacion;
    private final int noches;
    private final boolean desayuno;
    private final boolean cochera;
    private final boolean cuna;

    private Reserva(Builder builder) {
        if (builder.habitacion <= 0 || builder.noches <= 0) {
            throw new IllegalArgumentException("Habitacion y noches deben ser mayores a
cero");
        }
        this.habitacion = builder.habitacion;
        this.noches = builder.noches;
        this.desayuno = builder.desayuno;
        this.cochera = builder.cochera;
        this.cuna = builder.cuna;
    }

    public static class Builder {
        private final int habitacion;
        private final int noches;
        private boolean desayuno = false;
        private boolean cochera = false;
        private boolean cuna = false;

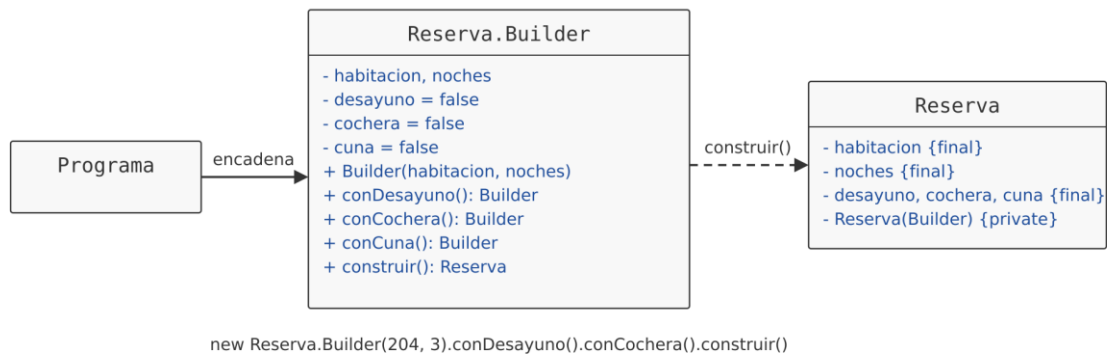
        public Builder(int habitacion, int noches) {
            this.habitacion = habitacion;
            this.noches = noches;
        }

        public Builder conDesayuno() { this.desayuno = true; return this; }
        public Builder conCochera() { this.cochera = true; return this; }
        public Builder conCuna() { this.cuna = true; return this; }

        public Reserva construir() {
            return new Reserva(this);
        }
    }
}
```

Una reserva se lee como se dice:

```
Reserva fin = new Reserva.Builder(204, 3)
    .conDesayuno()
    .conCochera()
    .construir();
```



El ejemplo con la estructura de Builder. Lo obligatorio entra por el constructor del Builder, y la Reserva se crea al final, validada y con todos sus atributos final.

Ventajas y desventajas

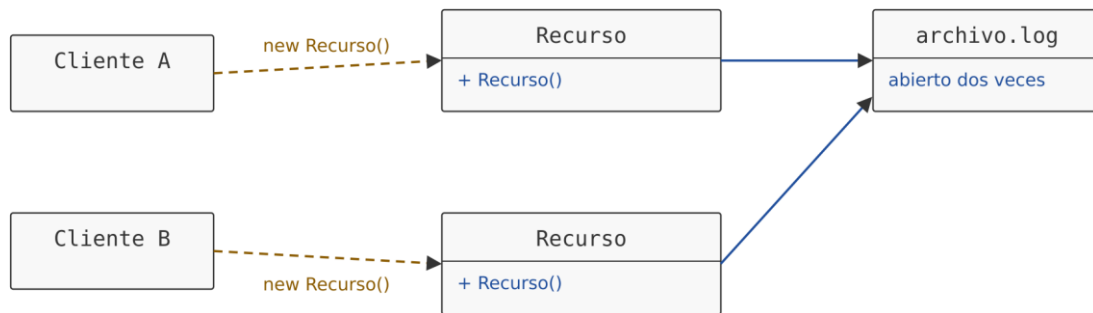
- El código que construye se lee, porque cada dato lleva el nombre de su método, y lo opcional se omite en vez de pasarse como *null*.
- Lo obligatorio no se puede olvidar, y el objeto se valida entero en un solo lugar antes de existir.
- El Producto puede ser inmutable, con todos sus atributos *final*, porque recibe todo junto.
- Duplica los atributos, una vez en el Builder y otra en el Producto, y agrega una clase. Para un objeto con tres datos es más de lo que hace falta.

Builder. Lo obligatorio en el constructor, lo opcional con nombre, y el objeto se valida entero en *construir()*.

Singleton

El problema

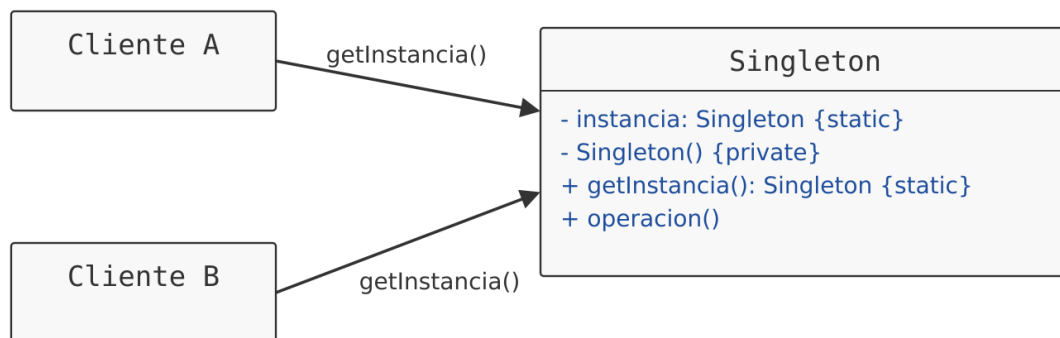
Algunos objetos no pueden existir dos veces. Un archivo de registro abierto por dos objetos distintos termina con líneas mezcladas. Una conexión a la base de datos abierta dos veces es un recurso desperdiciado o un error. Que el programa principal cree uno solo no alcanza, porque nada impide que otra clase escriba *new* y aparezca el segundo. La clase misma tiene que impedirlo.



Sin el patrón. El constructor es público, cada cliente escribe new, y el recurso que debía ser único aparece dos veces sobre el mismo archivo.

La solución

La clase controla su propia instancia con tres piezas. El constructor es privado, para que nadie afuera pueda escribir `new`. Un atributo estático guarda la única instancia. Y un método estático, por convención `getInstancia()`, la crea la primera vez que se lo llama y la devuelve todas las siguientes. Los miembros son *static* porque pertenecen a la clase y no a una instancia, y cuando se llama a `getInstancia()` por primera vez todavía no hay ninguna.



Estructura de Singleton. Nadie puede escribir new Singleton(). La clase guarda su única instancia y la entrega por getInstancia() a todos los clientes.

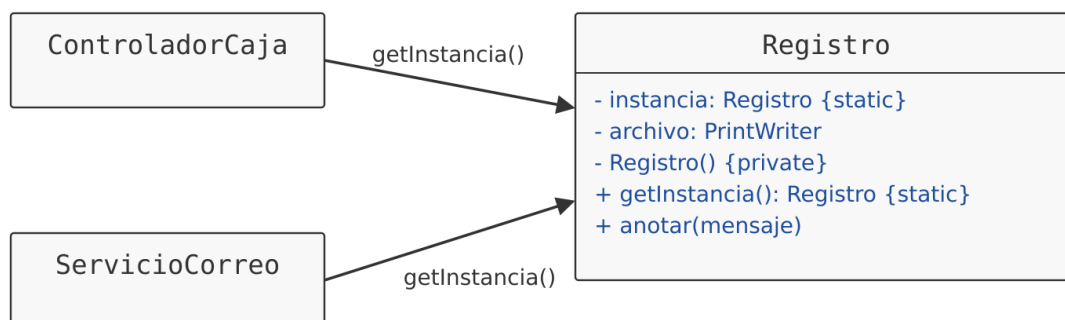
Ejemplo

Un registro de eventos escribe en un archivo lo que va pasando en la aplicación, con fecha y hora, y todas las clases tienen que escribir en el mismo:



```
public class Registro {  
  
    private static Registro instancia;  
    private final PrintWriter archivo;  
  
    private Registro() throws IOException {  
        this.archivo = new PrintWriter(new FileWriter("aplicacion.log", true));  
    }  
  
    public static Registro getInstancia() throws IOException {  
        if (instancia == null) {  
            instancia = new Registro();  
        }  
        return instancia;  
    }  
  
    public void anotar(String mensaje) {  
        this.archivo.println(LocalDate.now() + " " + mensaje);  
        this.archivo.flush();  
    }  
}
```

Cada llamada a `getInstancia()` en el programa devuelve el mismo objeto, y el archivo se abre una sola vez.



El ejemplo con la estructura de Singleton. Las dos clases escriben en el mismo archivo porque `getInstancia()` les entrega el mismo Registro.

Ventajas y desventajas

- La instancia es única por diseño, no por costumbre. Ninguna clase puede crear la segunda.
- Se crea recién cuando alguien la necesita, no al arrancar el programa.
- Es también un acceso global. Una llamada a `getInstancia()` escondida dentro de una clase es un acoplamiento con nombre y apellido, igual que un `new` de una clase concreta, y hace imposible probar esa clase sin el archivo o la base de verdad. Por eso muchos lo consideran un anti patrón cuando se usa para cualquier objeto “que hay uno solo”.



- Con varios hilos, dos llamadas simultáneas a *getInstance()* cuando *instancia* es *null* pasan las dos el *if* y crean dos objetos. La versión que lo evita se ve en la unidad de concurrencia.

La regla para este curso es una sola. *getInstance()* se llama en el programa principal, junto con los otros *new*, y el objeto viaja por constructor a quien lo necesite, como cualquier otra dependencia. Y Singleton se reserva para lo que de verdad no puede existir dos veces, como una conexión, un archivo abierto o el gestor de audio de un juego. “Una sola partida” o “un solo tablero” no son Singleton. Son uno porque el programa crea uno, y una prueba puede crear otro.

Singleton. Constructor privado, instancia estática y *getInstance()*. Uno por proyecto, y se inyecta igual que cualquier otra dependencia.

En la biblioteca

Los tres están en Java desde siempre. *Integer.valueOf(3)*, *List.of(“a”, “b”)* y *LocalDate.of(2026, 9, 28)* son métodos de creación al estilo Factory, un método estático que decide qué objeto devolver, y que a veces devuelve uno que ya existía. *StringBuilder* es un Builder, con *append()* devolviendo *this* para encadenar y *toString()* haciendo de *construir()*. *Runtime.getRuntime()* y, en Swing, *Toolkit.getDefaultToolkit()* son Singleton, con constructor privado y un método estático que siempre devuelve el mismo objeto.

Factory, Builder y Singleton son las tres maneras que tiene la biblioteca de no hacer escribir *new* a quien la usa.

Para el proyecto

Las tres preguntas de esta pre clase son dónde hay un *if* o un *switch* por tipo delante de un *new*, qué constructor tiene más de cuatro parámetros, y qué objeto no puede existir dos veces. La primera lleva a Factory o a Factory Method, según si la decisión viene de un dato o de la variante. La segunda lleva a Builder. La tercera lleva a Singleton, y conviene que la respuesta sea una sola clase. Las respuestas son la primera parte de la entrega de patrones.

Bibliografía sugerida para profundizar

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

- Capítulo 3, “Creational Patterns”, con “Factory Method”, “Builder” y “Singleton”. Es la descripción original, con el ejemplo del laberinto que recorre los cinco patrones creacionales.

Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2.^a ed.). O’Reilly.

- Capítulo 4, “The Factory Pattern”, que empieza por el Factory más simple antes de llegar a Factory Method, y capítulo 5, “The Singleton Pattern”, con el problema de los hilos explicado paso a paso.

Bloch, J. (2018). *Effective Java* (3.^a ed.). Addison-Wesley.



- Ítems 1, 2 y 3, sobre cuándo preferir un método estático de creación a un constructor, el Builder para constructores con muchos parámetros, y las formas de escribir un Singleton en Java.

Shvets, A. (2020). *Dive Into Design Patterns*. Refactoring.Guru. Versión web gratuita en <https://refactoring.guru/es/design-patterns>

- La sección “Patrones creacionales”, con el problema, la estructura, los diagramas y el código en Java de cada patrón. Es el material más visual de la lista y el mejor para empezar.